# Behaviour is text you can read

*Animated explainer · src/prompt.js*

A coding agent behaves the way it does because of three pieces of text. prompt.js owns the first: the standing rules, assembled at run time with a bounded memory digest. No prompt engine, no hidden CMS.

**7 scenes · 6 standing rules · 1 memory digest · rebuilt every task**

CONTENTS

# What is in this explainer


01 · Why this file exists ..... 3

02 · The standing rules ..... 4

03 · Rule 6 ..... 5

04 · Assembly ..... 6

05 · The digest ..... 7

06 · Why this is nice ..... 8

07 · Recap ..... 9


Page numbers refer to the folio at the foot of each page.

# 01 · Why this file exists

*Behaviour is three pieces of text.*

**System prompt** — How to work, and what the tools are for.

**Tool descriptions** — When to reach for each tool.

**Error strings** — How to recover from a wrong move.

> **remember** This file owns the first piece. Keeping it out of the loop is the point.
> **On screen:** A coding agent behaves the way it does because of three pieces of text. The system prompt is how to work. Tool descriptions say when. Error strings say how to recover. prompt.js owns the first piece, so agent.js can read as the loop rather than three screens of prose.

# 02 · The standing rules

*Six rules, in the order a run needs them.*

• 1 Look before you write. list_files / read_file before editing.

• 2 Prefer the right write. create_file, edit_file, write_file.

• 3 Copy old_string exactly. Every edit must match read_file output.

• 4 Run what you wrote. Read the error, fix it, run again.

• 5 Do not invent results. Stop when done and summarise.

• 6 Save durable lessons. remember facts, not task progress.

> Written in the second person, in the order a run needs them: look, edit, run, stop, remember.
> **On screen:** The rules are numbered for a reason. They follow the order a run actually works. Look first, choose the right write tool, copy old_string exactly, then run the result. Stop when done, and save only what will still be true next week.

# 03 · Rule 6

*Memory is a tool call, not task progress.*

**SRC/PROMPT.JS** · *source*

```
// If you learn something that will still be true next week -
// a command this repo needs, a preference the user stated,
// a trap you fell into - save it with remember.
// Never save task progress: "step 3 of 5" is not a memory,
// and it is worthless to the next run.
```

**Worth keeping** — “npm test needs Node 20.” “Never touch .env.”

**Not a memory** — “Step 3 of 5.” “File contents.” “A one-off command.”

> The note appears in the transcript like any other action, so saving is visible and reviewable.
> **On screen:** Rule six is the interesting one: memory is a tool call, not an internal side effect. Save facts that will still be true next week. Never save task progress. Because it is a tool call, everyone can see what was saved and why.

# 04 · Assembly

*Rules first. Memory second.*

**SRC/PROMPT.JS** · *source*

```
export function buildSystemPrompt({ memory = null, log = null } = {}) {
  const rules = systemRules();
  if (!memory) return rules;

  const digest = memory.digest();
  if (!digest) return rules;

  const injected = digest.split('\n').filter((line) => line.startsWith('- [')).length;
  log?.(ui.memory(memory.label + ' - ' + injected + ' memories added to the system prompt'));

  return rules + '\n\n' + digest;
}
```

**BEFORE STEP 1** · *text*

```
memory ← src/memory.md - 2 memories added to the system prompt
```

> **On screen:** buildSystemPrompt starts with the standing rules. If there is no memory, that is the whole system message. If there is a digest, it is appended, and the log says how many memories were carried.

# 05 · The digest

*Bounded twice. Honest about what is hidden.*

**THE MEMORY BLOCK** · *text*

```
## What you remember from earlier runs
These notes live in src/memory.md and were learned before this task started.
Treat them as strong hints, not facts to trust blindly: the code may have moved on since.
- [fact] npm test needs Node 20. _(2026-10-20)_
- [gotcha] .env must never be edited by the agent. _(2026-10-22)_
(3 older memories not shown - search src/memory.md with the recall tool)
```

The digest is bounded by entry count and characters, and it names the lazy path: recall.
**remember** A bounded prompt is what stops memory from quietly eating the context window.
**On screen:** The digest is a small markdown block, not the whole memory file. It is capped by both entry count and characters, so it cannot grow without bound. If older memories were left out, the block says so and points at recall.

# 06 · Why this is nice

*Prompt engineering you can diff.*

- 📝 No template engine the prompt is a function of memory and tool names.
- 🗒️ No runtime file the rules are in this module, not a CMS.
- 🔍 No hidden variables everything that changes the prompt is an argument.

> **remember** A prompt you can read in the repository is a prompt you can review, test and change deliberately.
> **On screen:** The prompt is not loaded from a database or assembled by a template engine. It is a function of two things: the tool names and the memory digest. That makes it reviewable. A prompt change is a normal code change.

# 07 · Recap

*The rules are part of the program.*

• 6 standing rules

• 1 memory digest

• 0 prompt engines

→

**Rules first.** The model learns how to work before it learns what is remembered.

→

**Memory second.** Append only when there is something useful and bounded.

→

**Save durable facts.** Never task progress.

→

**Keep it diffable.** A prompt is code you can review.

> **remember** The system prompt is not magic. It is prose with a clear owner.
> **On screen:** Six rules, one memory digest, and no prompt engine. Rules first, memory second, durable facts only, and keep it diffable. That is how the agent knows how to work.